



## **POSLink2 Migration Overview**

04-24-2025

V1.00

## Preface

Copyright © 2025 PAX Technology, Inc. All rights reserved.

The Programs (which include both the software and documentation) contain proprietary information; they are provided under a license agreement containing restrictions on use and disclosure and are also protected by copyright, patent, and other intellectual and industrial property laws. Reverse engineering, disassembly, or decompiling of the Programs, except to the extent required to obtain interoperability with other independently created software or as specified by law, is prohibited.

This document is provided for informational purposes only. All features and specifications are subject to change without notice. If there are any problems in the documentation, please report them to PAX in writing. This document is not warranted to be error-free. Except as may be expressly permitted in the license agreement for these Programs, no part of these Programs may be reproduced or transmitted in any form or by any means, electronic or mechanical, for any purpose.

Security Notice: No part of this publication may be copied, distributed, stored in a retrieval system, translated into any human or computer language, transmitted, in any form or by any means, without the prior written consent of PAX Technology, Inc.

PAX is a registered trademark of PAX Technology Limited in China and/or other countries. All other trademarks or brand names are the properties of their respective holders. PAXBiz, PAXSTORE, POSDK, The PAX Portfolio Manager, and/or other PAX products referenced herein are either trademarks or registered trademarks of PAX Technology, Inc. or its Affiliates. Other product and company names mentioned herein may be trademarks of their respective owners.

The *Bluetooth*® word mark and logos are registered trademarks owned by Bluetooth SIG, Inc., and any use of such marks by PAX Technology, Inc. is under license. Other trademarks and trade names are those of their respective owners.

EMV® is a registered trademark in the U.S. and other countries and an unregistered trademark elsewhere. The EMV trademark is owned by EMVCo, LLC.

Android is a trademark of Google LLC.

## TECHNICAL SUPPORT

If there is a problem while installing, registering, or operating this product, please make sure to review the documentation. If unable to resolve the issue, please contact PAX.

Monday-Friday 9:00 AM to 1:00 AM  
Saturday 9:00 AM to 5:00 PM  
Sunday Closed

The level of access to this Service is by the support plan arrangements made between PAX and the Organization. Please consult this support plan for further information about entitlements, including the hours when telephone support is available.

## TECHNICAL SUPPORT CONTACT INFORMATION

Phone: (877) 859-0099  
Email: [support@pax.us](mailto:support@pax.us)  
URL: [www.pax.us](http://www.pax.us)

## Revision History

| Date       | Version | Description      |
|------------|---------|------------------|
| 04-24-2025 | v1.0    | Initial release. |

## Table of Contents

|                                                             |           |
|-------------------------------------------------------------|-----------|
| <b>1. Purpose</b>                                           | <b>1</b>  |
| <b>2. Intended Audience</b>                                 | <b>1</b>  |
| <b>3. Notable Feature Differences</b>                       | <b>1</b>  |
| 3.1 Compatibility with POSLink1                             | 1         |
| 3.2 Elimination of Low-Level Data Types                     | 1         |
| 3.3 Developer-Friendly Architecture & Split Functionality   | 1         |
| 3.4 POSLink1 vs. POSLink2 Comparison                        | 2         |
| <b>4. Key Structural Changes</b>                            | <b>2</b>  |
| 4.1 POSLink1                                                | 2         |
| 4.2 POSLink2                                                | 3         |
| 4.2.1 Semi-Integration                                      | 3         |
| 4.2.2 Full-Integration                                      | 4         |
| 4.2.3 Peripherals                                           | 5         |
| <b>5. Smooth Migration Path</b>                             | <b>6</b>  |
| 5.1 Step 1 - Choose Your Target SDK                         | 6         |
| 5.2 Step 2 - Understand the Structural Changes              | 7         |
| 5.3 Step 3 - Get Familiar with the New SDK                  | 7         |
| 5.4 Step 4 - Consult the API Guide                          | 7         |
| 5.5 Example - Migration from POSLink1 to POSLink2           | 7         |
| <b>6. Major Function Changes</b>                            | <b>7</b>  |
| 6.1 Payment Functions                                       | 7         |
| 6.1.1 Deprecation of Passthru Data                          | 7         |
| 6.1.2 DoEBT Request & Response Overhaul                     | 10        |
| 6.1.3 Force Commercial Card Flag Transition                 | 10        |
| 6.1.4 Adding Signature Capture and Retrieval                | 10        |
| 6.1.5 FPS Response Handling and Structural Changes          | 11        |
| 6.1.6 Tokenization and Card on File Enhancements            | 12        |
| 6.1.7 TOR Response Enhancements                             | 13        |
| 6.1.8 Host Trace Information Integration                    | 13        |
| 6.1.9 Host Gateway Renamed to Host Information Request      | 15        |
| 6.1.10 Relocating Fields from the Original Class            | 16        |
| 6.2 Non-Payment Functions                                   | 19        |
| 6.2.1 Command and Field Renaming Updates                    | 19        |
| 6.2.2 ShowTextBox Command - Hard Key List Field Type Change | 20        |
| 6.2.3 SAF Parameter Updates                                 | 20        |
| <b>7. PAX Customer Support</b>                              | <b>23</b> |

## 1. Purpose

As we identified limitations in the original structure of POSLink1, PAX is introducing POSLink2 as an enhanced replacement. POSLink2 features a modernized, modular design with streamlined capabilities to simplify development and integration. While POSLink2 maintains compatibility with POSLink1 within the same project, this article is designed to help our valued customers transition smoothly by outlining the key changes.

After reading this, you will gain a high-level understanding of the difference between POSLink1 and POSLink2, including its structural improvements. For technical details, please refer to our API Guide, Integration Guide, or open a ticket on our developer platform, [Quail](#).

## 2. Intended Audience

This document is intended for developers who have previously worked with POSLink1 and are now transitioning to POSLink2.

## 3. Notable Feature Differences

### 3.1 Compatibility with POSLink1

- POSLink2 maintains compatibility with POSLink1 within the same project, ensuring a smoother transition for existing customers.  
⚠ For Java users: Please ensure POSLink1 is updated to version 1.13.00 or later to avoid potential class duplication issues when using both SDKs together.

### 3.2 Elimination of Low-Level Data Types

POSLink2 moves away from using low-level data types (e.g., integers, strings) and replaces them with more structured objects and fields.

- **ExtData:** The XML field "ExtData" no longer exists; it has been replaced by objects and fields for better data handling.
- **Enums:** Enums are now used to define available options, making them visible in the IDE as developers' type, improving the developer experience.

### 3.3 Developer-Friendly Architecture & Split Functionality

The changes made to the architecture and functionality serve the following high-level purposes:

- **Modularization:** Improving maintainability and scalability by breaking up the SDK into smaller, focused components.
- **Simplified Integration:** Separating functionality based on integration needs (Semi and Full) and sharing core functions through a common Admin library. For more detailed explanations of these changes, please refer to the [Key Structural Changes](#) section below.

## 3.4 POSLink1 vs. POSLink2 Comparison

Here's a quick comparison highlighting the key differences between POSLink1 and POSLink2:

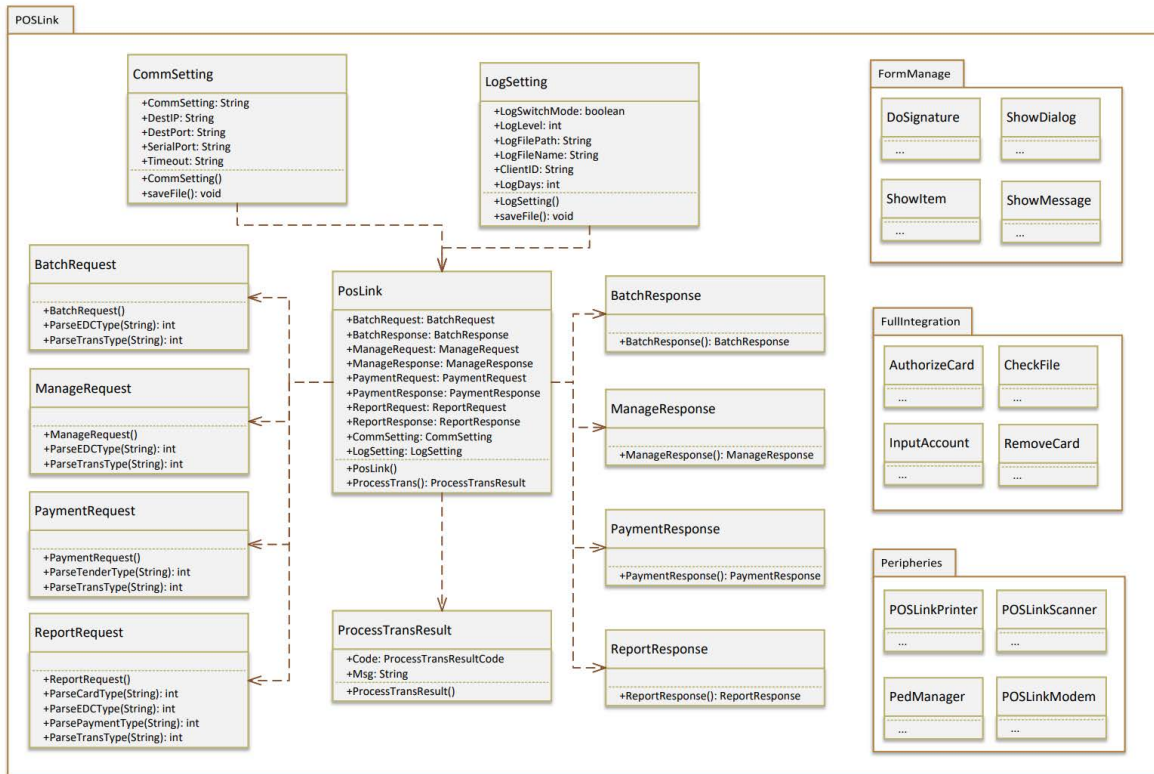
| Aspect                      | POSLink1                                                                                                              | POSLink2                                                                                                                                           |
|-----------------------------|-----------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Architectural Design</b> | <b>Coupled:</b> All features integrated into one SDK, leading to a bloated SDK that is difficult to integrate.        | <b>Decoupled:</b> Split into multiple SDKs based on business functions, including Semi-integration SDK, Full-integration SDK, and Peripherals SDK. |
| <b>API Design</b>           | <b>Mixed:</b> Multiple commands share an interface, and multiple properties share a class.                            | <b>Cleared:</b> One command, one interface. Classes and enumerations are widely used to improve clarity.                                           |
| <b>Business Notes</b>       | <b>Missed:</b> Interface descriptions are missing, making understanding difficult without referring to the API Guide. | <b>Exhaustive:</b> Detailed interface and parameter notes are available, making understanding easier directly through the IDE.                     |
| <b>Quick Guide</b>          | <b>Missed:</b> No quick guide available.                                                                              | <b>Fully Supported:</b> The quick guide is included to help developers quickly understand and begin integration projects.                          |
| <b>Demo UI</b>              | <b>Mixed:</b> Multiple requests sent on one page, which can make it hard to separate parameters.                      | <b>Cleared:</b> One command per page, making the demo more intuitive and clearer.                                                                  |

## 4. Key Structural Changes

### 4.1 POSLink1

In the original POSLink1 SDK structure, all functionalities were consolidated within a single, large POSLink package. This design often led to unnecessary storage usage—for example, users leveraging only semi-integration features would still inherit the full set of APIs, including those related to full integration, and vice versa.

Additionally, all transaction types shared a single request class, increasing the risk of incorrect parameter usage. For instance, when initiating a Credit Sale, the developer would need to manually identify and input the corresponding transaction code. Parameters irrelevant to the selected transaction type could also be mistakenly included, leading to integration errors or unpredictable behavior.



Class Diagram of POSLink1

## 4.2 POSLink2

In the new POSLink2 SDK architecture, functionalities are modularized by purpose rather than grouped into a single monolithic package. This structured design improves clarity, efficiency, and maintainability.

We have separated the SDK into three distinct packages based on integration purpose: **Semi-Integration**, **Full-Integration**, and **Peripherals**. Each SDK is tailored for its specific use case, allowing developers to work with a focused set of tools.

This modular setup enables teams to include only the components relevant to their integration type, reducing application size and minimizing the risk of misusing unrelated functions.

### 4.2.1 SEMI-INTEGRATION

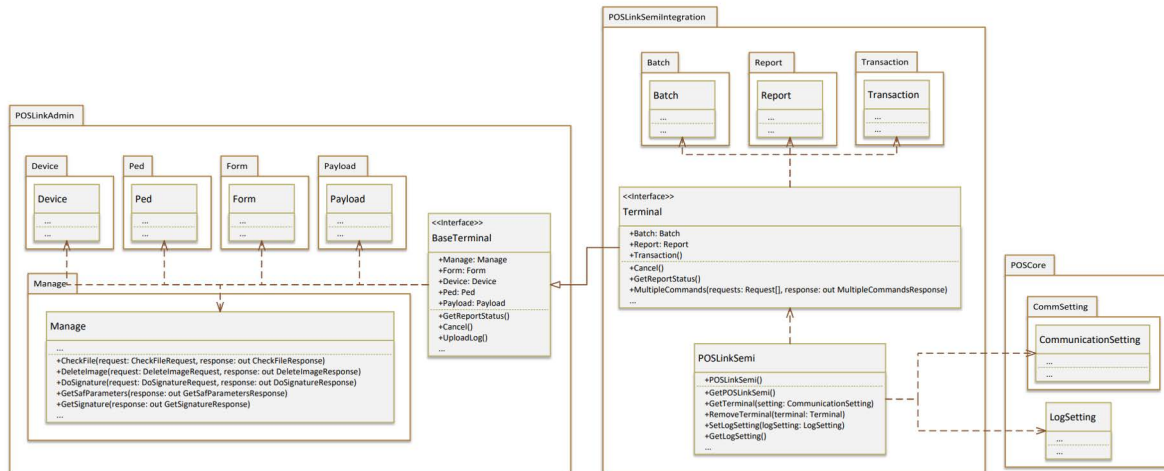
The POSLink2 Semi-Integration SDK is designed with clear modular boundaries to enhance flexibility and maintainability. The following class diagram illustrates the overall structure of the Semi-Integration design, which includes three key packages:

- **POSLink Core:** Handles core functionalities such as communication setup and log configuration.
- **POSLink Admin:** Shared by both integration types, it includes common

components related to device control, PED, forms, payloads, and system management.

- **POSLink Semi-Integration:** Contains features specific to the Semi-Integration approach, including transaction request/response handling and session control.

**Note:** The iOS SDK does not include a separate POSLink Core package. For platform-specific details, refer to the POSLink2 iOS Semi-Integration Guide.



*Class Diagram of POSLink2 Semi-Integration*

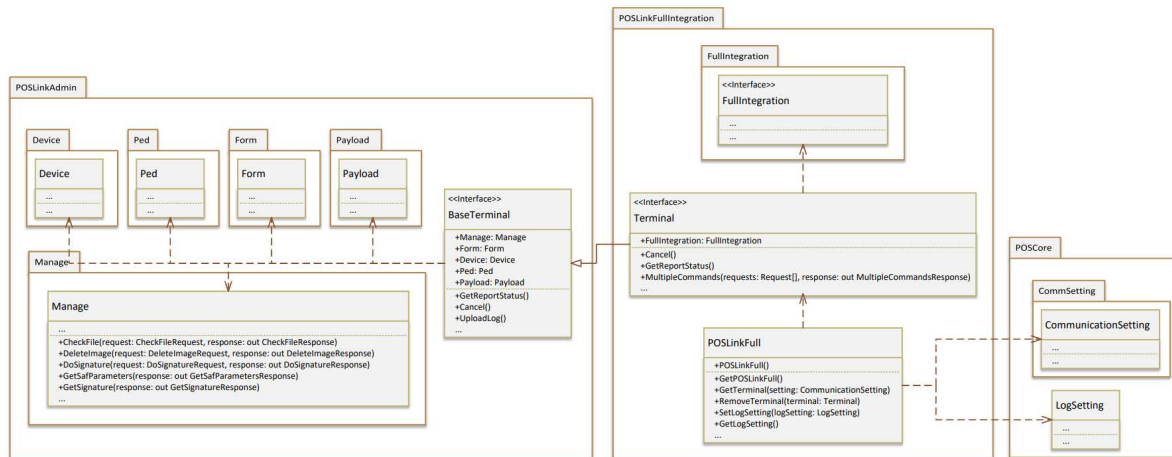
## 4.2.2 FULL-INTEGRATION

The POSLink2 Full-Integration SDK follows a modular structure similar to the Semi-Integration design. It is organized into three primary packages to support clear separation of concerns and ease of use:

- **POSLink Core:** Manages essential operations such as communication initialization and log settings.
- **POSLink Admin:** A shared package used by both integration types, providing common features for device control, PED, forms, payloads, and system management.
- **POSLink Full-Integration:** Contains functions tailored for Full-Integration workflows, including deeper transaction control and embedded payment logic.

**Note:** The iOS SDK does not include a separate POSLink Core package. For platform-specific details, refer to the POSLink2 iOS Full-Integration Guide.

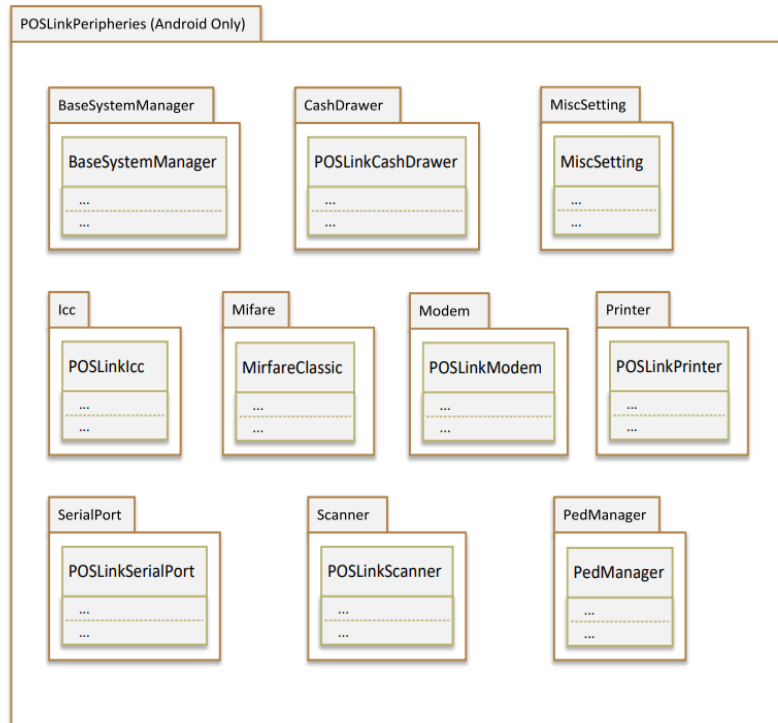




Class Diagram of POSLink2 Full-Integration

## 4.2.3 PERIPHERIES

The POSLink Peripherals package is dedicated to peripherals such as printers, scanners, and cash drawers. It has been separated from the grouped package and is designed specifically for PAX Android devices.



Class Diagram of POSLink2 Peripherals (Android Only)

## 5. Smooth Migration Path

To simplify your transition to POSLink2, follow these structured steps to identify the correct SDK, understand structural differences, and leverage our documentation effectively.

### 5.1 Step 1 - Choose Your Target SDK

POSLink2 separates functionality across different SDKs based on platform and integration type. Refer to the following table to determine the appropriate SDK for your project:

| Platform                                        | Integration Type                  | Recommended Library                     |
|-------------------------------------------------|-----------------------------------|-----------------------------------------|
| POSLink iOS/MacOS                               | BroadPOS Host Application         | POSLink2 iOS/MacOS Semi Integration     |
|                                                 | TermLink or BroadPOS P2PE         | POSLink2 iOS/MacOS Full Integration     |
| POSLink .NET Standard or POSLink .NET Framework | BroadPOS Host Application         | POSLink2 .NET Standard Semi Integration |
|                                                 | TermLink or BroadPOS P2PE         | POSLink2 .NET Standard Full Integration |
| POSLink Java/Android                            | BroadPOS Host Application         | POSLink2 Java/Android Semi Integration  |
|                                                 | TermLink or BroadPOS P2PE         | POSLink2 Java/Android Full Integration  |
|                                                 | Peripherals (Scan, Print, MIFARE) | POSLink2 Peripheries                    |

## 5.2 Step 2 - Understand the Structural Changes

After selecting your target SDK, refer to the [Key Structural Changes](#) section in this document. This will help you understand how the SDK is organized in POSLink2 compared to POSLink1.

## 5.3 Step 3 - Get Familiar with the New SDK

Each POSLink2 SDK includes a **README** file that points to the appropriate integration guide. Use this guide to become familiar with the SDK structure and implementation flow.

## 5.4 Step 4 - Consult the API Guide

For details on individual classes and functions, refer to the **API Guide** located in the Doc folder of your SDK package. This will help answer specific implementation questions.

## 5.5 Example - Migration from POSLink1 to POSLink2

For example, if a user is migrating from the **POSLink1 Java SDK** and only uses the **BroadPOS Host Application with a Semi-Integration solution**, they would follow these steps:

- **Step 1:** Choose **POSLink2 Java/Android Semi-Integration SDK**.
- **Step 2:** Refer to the **Key Structural Changes** section for insights on differences between POSLink1 and POSLink2 Semi-Integration.
- **Step 3:** Open the SDK folder and review the README file, which links to the **POSLink2 Semi-Integration Guide for Java and Android**.
- **Step 4:** Navigate to the Doc folder and use the **API Guide** for Java/Android to review function-level details and examples.

## 6. Major Function Changes

### 6.1 Payment Functions

#### 6.1.1 DEPRECATION OF PASSTHRU DATA

To make our SDK more streamlined and high-level, support for **Passthru** Data has been removed. If you previously used this field, we now offer several alternative solutions to suit your needs. Please refer to the updated API documentation for detailed guidance.

##### 6.1.1.1 Fleetcard

To process **Fleet** transactions, use the `setFleetCard(FleetCardRequest fleetCard)` method in your **DoCreditRequest**.

```
// Create a new instance of the FleetCardRequest object
FleetCardRequest fleetCardRequest = new FleetCardRequest();

// Set the customer data in the FleetCardRequest
fleetCardRequest.setCustomerData("example_123");

// Set the job number associated with the FleetCard request
fleetCardRequest.setJobNumber("123456789");

// Set the vehicle ID for the FleetCard request
fleetCardRequest.setVehicleId("123456789");

// Associate the FleetCardRequest with the DoCreditRequest object
doCreditRequest.setFleetCard(fleetCardRequest);
```

## 6.1.1.2 FSA

We have added **FsaData** as a member of the **Credit** and **Debit** transaction request messages. To set FSA data for a specific transaction request, use the `setFsaData()` method in your **DoCreditRequest**.

```
// Create a new DoCreditRequest object
DoCreditRequest doCreditRequest = new DoCreditRequest();

// Create a new FsaData object to store FSA-related information
FsaData fsaData = new FsaData();

// Set the FSA data in the DoCreditRequest
doCreditRequest.setFsaData(fsaData); // Associates FSA data with the
credit request
```

Additionally, the **Force FSA** has been moved from `TransactionBehavior` to `FsaData`. To set the **Force FSA flag**, use the `setForceFsa()` method in **FsaData**.

```
// Create a new FsaData object for Force FSA functionality
FsaData fsaData = new FsaData();

// Set the Force FSA flag value
fsaData.setForceFsa("ForceFsa.YES");
```

## 6.1.1.3 eWIC

We have created **eWIC**, added the **eWIC Request** to the **Do EBT Request API**, and introduced the **eWIC Response** to the **Do EBT Response API**.

To create a new **EwicRequest** and set its values, follow these steps:

```
// Create a new EwicRequest object to hold the eWIC request data
EwicInformationRequest ewicInformation = new
EwicInformationRequest();

// Create a new EwicData object to store the specific eWIC data
EwicData ewicData = new EwicData();

// Set the eWIC data in the eWIC request
ewicInformation.setEwicData(ewicData);

// Set the eWIC discount amount
ewicInformation.setEwicDiscountAmount("000001500");

// Create a new DoEbtRequest object to process the EBT request
DoEbtRequest doEbtRequest = new DoEbtRequest();

// Set the eWIC information in the EBT request
doEbtRequest.setEwicInformation(ewicInformation);
```

To retrieve data from the **eWIC response**, follow these steps:

```
// Retrieve the eWIC information from an existing DoEbtResponse
object
EwicInformationResponse ewicInformation =
doEbtResponse.ewicInformation();

// Retrieve the eWIC balance
List<EwicBalance> ewicBalance = ewicInformation.ewicBalance();

// Retrieve specific details about the eWIC
List<EwicDetail> ewicDetail = ewicInformation.ewicDetail();

// Retrieve the earliest benefit expiry date from the eWIC response
String earliestBenefitExpiryDate =
ewicInformation.earliestBenefitExpiryData();
```

## 6.1.2 DoEBT REQUEST & RESPONSE OVERHAUL

The **DoEBT** Request and Response have been streamlined as part of the improvements to simplify transaction handling. Moved **continuousScreen** to **TransactionBehavior Request** for better transaction management.

```
// TransactionBehavior object is created to hold transaction-related settings
TransactionBehavior transactionBehavior = new TransactionBehavior();

// Set the continuous screen flag in the TransactionBehavior object
// This setting is now managed under TransactionBehavior instead of DoEBT request
transactionBehavior.setContinuousScreen("ContinuousScreen.DETAULT");

// Set the updated TransactionBehavior to the DoEBT Request
doEbtRequest.setTransactionBehavior(transactionBehavior);
```

## 6.1.3 FORCE COMMERCIAL CARD FLAG TRANSITION

The **Force Commercial Card flag** is now set in the **CommercialRequest** object, rather than in TransactionBehavior.

```
// Create a CommercialRequest object to handle commercial card-related information
CommercialRequest commercialRequest = new CommercialRequest();

// Set the Force Commercial Card flag in the CommercialRequest object
// This flag now resides in the CommercialRequest instead of TransactionBehavior
commercialRequest.setForceCc("ForceCc.YES");
```

## 6.1.4 ADDING SIGNATURE CAPTURE AND RETRIEVAL

To incorporate **signature details** into a transaction, use the `setSignatureInformation(SignatureRequest signatureInformation)` method within **DoCreditRequest**. Upon receiving a response, retrieve the **signature data** using `signatureInformation()` within **DoCreditResponse**. This process applies to **all EDC types**.

```
// Create a DoCreditRequest object for processing a credit transaction
DoCreditRequest doCreditRequest = new DoCreditRequest();

// Create a SignatureRequest object to configure signature settings
SignatureRequest signatureRequest = new SignatureRequest();

// Enable signature capture during the transaction
signatureRequest.setSignatureCaptureFlag(SignatureCaptureFlag.CAPTURE);

// Set the signature acquisition flag to REQUIRED, ensuring a signature is collected
signatureRequest.setSignatureAcquireFlag(SignatureAcquireFlag.REQUIRED);

// Attach the signature request settings to the credit request
doCreditRequest.setSignatureInformation(signatureRequest);

// After the transaction response is received, retrieve the captured signature data
String signData =
doCreditResponse.signatureInformation().signatureData();
```

## 6.1.5 FPS RESPONSE HANDLING AND STRUCTURAL CHANGES

We have restructured Payment Transaction Information by categorizing different sections, including FPS information. To enhance clarity, we introduced the **FPS Response** group and integrated it into the **Credit/Debit Response** and **Local Detail Report Response**. Some fields previously part of Response Transaction Information have been moved to FPS Response.

```
// Retrieve FPS Response from an existing DoCreditResponse
FpsResponse fpsResponse = doCreditResponse.fpsInformation();

// Extract FPS-related details from the response
String fps = fpsResponse.fps(); // Retrieve FPS value

// Retrieve FPS signature
String fpsSignature = fpsResponse.fpsSignature();

// Retrieve FPS receipt data
FpsReceipt fpsReceipt = FpsReceipt.valueOf(fpsResponse.fpsReceipt());
```

## 6.1.6 TOKENIZATION AND CARD ON FILE ENHANCEMENTS

To enhance security and streamline **tokenized** transactions, we introduced **CofRequest** and **CofResponse**. Some fields have been moved to these new structures to improve clarity and usability.

| Type        | Field Name                    |
|-------------|-------------------------------|
| CofRequest  | tokenRequestFlag              |
|             | cofIndicator                  |
|             | cofInitiator                  |
|             | token                         |
|             | originalTransactionIdentifier |
|             | tokenSerialNumber             |
|             | originalPaymentService2000    |
|             | originalAuthorizationResponse |
| CofResponse | token                         |
|             | tokenSerialNumber             |
|             | transactionId                 |
|             | paymentService2000            |
|             | authorizationResponse         |

Requesting a Token:

```
// Create a CofRequest object for tokenization
CofRequest cofRequest = new CofRequest();

// Set the token request flag to indicate a token should be generated
cofRequest.setTokenRequestFlag(TokenRequestFlag.YES);

// Attach CofRequest to the credit request
doCreditRequest.setCofInformation(cofRequest);

// After the transaction response is received, retrieve CofResponse
// from the DoCreditResponse
CofResponse cofResponse = doCreditResponse.cofResponse();

// Extract the token value from the response
String token = cofResponse.token();
```



Processing a Transaction with a Token:

```
// Create a CofRequest object for processing a tokenized transaction
CofRequest cofRequest = new CofRequest();

// Set the token value
cofRequest.setToken("your_token_here");

// Specify that the transaction uses a stored card
cofRequest.setCofIndicator(CofIndicator.YES);

// Indicate the transaction initiator
cofRequest.setCofInitiator(CofInitiator.CUSTOMER);

// Attach CofRequest to the credit request
doCreditRequest.setCofInformation(cofRequest);
```

## 6.1.7 TOR RESPONSE ENHANCEMENTS

We have added **reversalStatus** and **reverseAmount** to the TOR Response Information to provide better insight into transaction reversals. These fields help developers track whether a transaction has been reversed and the amount involved.

After the transaction response has been successfully completed, you can access the reversal details as follows:

```
// Create a TOR response object
TorResponse torResponse = new TorResponse();

// Retrieve the status of the reversal
String reversalStatus = torResponse.reversalStatus();

// Retrieve the amount that was reversed
String reverseAmount = torResponse.reverseAmount();
```

## 6.1.8 HOST TRACE INFORMATION INTEGRATION

We have introduced **hostTraceInformation**, which consolidates elements from the **host response**, **trace request**, and **trace response**. To support this, we have added **hostTraceRequest** to **all Transaction Request APIs** and **hostTraceResponse** to **all Transaction Response APIs**.

Setting Host Trace Information in a Transaction Request:

```
// Create a host trace request object
HostTraceRequest hostTraceRequest = new HostTraceRequest();

// Set the ECR transaction ID
hostTraceRequest.setEcrTransactionId("erc_transaction_id");

// Set the original trace number
hostTraceRequest.setOriginalTraceNumber("original_trace_number");

// Set the gateway ID
hostTraceRequest.setGatewayId("gateway_id");

// Set the host reference number
hostTraceRequest.setHostReferenceNumber("host_refernece_number");

// Set the settlement date
hostTraceRequest.setSettlementDate("20001231");

// Set the batch number
hostTraceRequest.setBatchNumber("batch_number");

// Attach the host trace request to the credit request
doCreditRequest.setHostTraceInformation(hostTraceRequest);
```

Retrieving Host Trace Information from a Response:

```
// Retrieve host trace response
HostTraceResponse hostTraceResponse =
doCreditResponse.getHostTraceInformation();

// Retrieve ECR transaction ID
String ecrTransactionId = hostTraceResponse.ecrTransactionId();

// Retrieve host timestamp
String hostTimeStamp = hostTraceResponse.hostTimeStamp();

// Retrieve host reference number
String hostReferenceNumber = hostTraceResponse.hostReferenceNumber();

// Retrieve host trace number
String hostTraceNumber = hostTraceResponse.hostTraceNumber();
```

## 6.1.9 Host Gateway Renamed to Host Information Request

We have **renamed HostGateway to Host Information Request** and restructured its data handling. Some fields, such as **token**, have been removed, while **Authorization Code** has been moved from **Trace Request** to **Host Information Request**, along with **Pos Echo Data** from **Transaction API**.

Setting Host Information in a Transaction Request:

```
// Create a host information request object
HostInformationRequest hostInformationRequest = new
HostInformationRequest();

// Set return reason
hostInformationRequest.setReturnReason("HostGatewayReturnReason.GENER
AL_RETURN");

// Set custom data fields
hostInformationRequest.setCustomizeData1("example_custmize_data_1");
hostInformationRequest.setCustomizeData2("example_custmize_data_2");
hostInformationRequest.setCustomizeData3("example_custmize_data_3");

// Set authorization code
hostInformationRequest.setAuthorizationCode("authorization_code");

// Set POS echo data
hostInformationRequest.setPosEchoData("example_pos_echo_data");

// Attach the host information request to the credit request
doCreditRequest.setHostInformationRequest(hostInformationRequest);
```

Retrieving Host Information from a Response:

```
// Retrieve host information response
HostInformationResponse hostInformationResponse =
doCreditResponse.hostInformation();

// Retrieve return reason
HostGatewayReturnReason returnReason =
HostGatewayReturnReason.valueOf(hostInformationResponse.returnReason(
));

// Retrieve custom data fields
String customizeData1 = hostInformationResponse.customizeData1();
String customizeData2 = hostInformationResponse.customizeData2();
String customizeData3 = hostInformationResponse.customizeData3();

// Retrieve authorization code
String authorizationCode =
hostInformationResponse.authorizationCode();

// Retrieve POS echo data
String posEchoData = hostInformationResponse.posEchoData();
```

The following fields have been **moved to other groups or removed**:

**tokenRequestFlag, token, tokenSerialNumber, cardType, stationId, globalUid, ewicDiscountAmount**

## 6.1.10 RELOCATING FIELDS FROM THE ORIGINAL CLASS

Fields previously grouped under the **Original** Class in Transaction APIs have been redistributed into more relevant classes. This change improves organization and clarity. To preserve context, these fields now carry the prefix “original” in their new locations.

| Field           | Relocated to                              |
|-----------------|-------------------------------------------|
| transactionDate | Request Trace Information                 |
| transactionTime | Request Trace Information                 |
| settlementDate  | Host Trace Information (Request/Response) |
| batchNumber     | Host Trace Information (Request/Response) |
| pan             | Request Account Information               |
| expiryDate      | Request Account Information               |

| Field                 | Relocated to               |
|-----------------------|----------------------------|
| amount                | Request Amount Information |
| transactionType       | Each Transaction API       |
| paymentService2000    | Request COF                |
| authorizationResponse | Request COF                |

## 1. Storing Transaction Date & Transaction Time in Request Trace Information

```
// Create a trace request object
TraceRequest traceRequest = new TraceRequest();

// Set original transaction date
traceRequest.setOriginalTransactionDate("20001231");

// Set original transaction time
traceRequest.setOriginalTransactionTime("235959");
```

## 2. Storing Settlement Date & Batch Number in Host Trace Information

To set values in a Host Trace request:

```
// Create a host trace request object
HostTraceRequest hostTraceRequest = new HostTraceRequest();

// Set original settlement date
hostTraceRequest.setOriginalSettlementDate("20001231");

// Set original batch number
hostTraceRequest.setOriginalBatchNumber("123456789");
```

To retrieve data from the Host Trace response:

```
// Retrieve host trace information from the credit response
HostTraceResponse hostTraceInformation =
doCreditResponse.getHostTraceInformation();

// Retrieve original settlement date
String settlementDate =
hostTraceInformation.originalSettlementDate();

// Retrieve original batch number
String batchNumber = hostTraceInformation.originalBatchNumber();
```

### 3. Storing PAN & Expiry Date in Request Account Information

```
// Create an account request object
AccountRequest accountRequest = new AccountRequest();

// Set original PAN
accountRequest.setOriginalPan("1234");

// Set original expiry date
accountRequest.setOriginalExpiryDate("1226");

// Set original transaction amount
accountRequest.setOriginalAmount("000001500");
```

### 4. Storing Amount in Request Amount Information

```
// Create an amount request object
AmountRequest amountRequest = new AmountRequest();

// Set original transaction amount
amountRequest.setOriginalAmount("000001500");
```

## 5. Storing Transaction Type in Each Transaction API

```
// Create a credit request object
DoCreditRequest doCreditRequest = new DoCreditRequest();

// Set original transaction type
doCreditRequest.setOriginalTransactionType(TransactionType.SALE);
```

## 6. Storing Payment Service 2000 & Authorization Response in Request COF

```
// Create a COF request object
CofRequest cofRequest = new CofRequest();

// Set original payment service 2000
cofRequest.setOriginalPaymentService2000("original_payment_service_2000");

// Set original authorization response
cofRequest.setOriginalAuthorizationResponse("original_authorization_response");
```

## 6.2 Non-Payment Functions

### 6.2.1 COMMAND AND FIELD RENAMING UPDATES

The commands **ResetMsr** and **Reset** have been renamed to **ClearCardBuffer** and **ResetScreen** respectively, and the field **itemIndex** has been redefined as an **array** and renamed to **itemIndices** in the **ShowItem** and **ShowMessage** commands.

| Change Category | Old Name                 | New Name            |
|-----------------|--------------------------|---------------------|
| Command Rename  | ResetMsr                 | ClearCardBuffer     |
| Command Rename  | Reset                    | ResetScreen         |
| Field Update    | itemIndex (single value) | itemIndices (array) |

## 1. ResetMsr → ClearCardBuffer

The ResetMsr command has been **renamed** to **ClearCardBuffer**.

## 2. Reset → ResetScreen

The Reset command has been **renamed** to **ResetScreen**.

## 3. ShowItem & ShowMessage – itemIndex → itemIndices (Array Format)

itemIndex has been redefined as an **array** and **renamed** to itemIndices. Affects both **ShowItem** and **ShowMessage** commands.

### 6.2.2 SHOWTEXTBOX COMMAND - HARD KEY LIST FIELD TYPE CHANGE

We updated the **ShowTextBox** command by changing the **hardKeyList** field type from **String** to **Array**. This allows multiple hard key values to be set instead of a single string, improving flexibility when defining valid input keys.

```
// Create a ShowTextBox request object
ShowTextBoxRequest showTextBoxRequest = new ShowTextBoxRequest();

// Before: Set hardKeyList as a single String (no longer supported)
showTextBoxRequest.setHardKeyList("hard_key_list");

// After: Define hardKeyList as an array and assign multiple values
String[] hardKeyList = new String[2];
showTextBoxRequest.setHardKeyList(hardKeyList);
```

### 6.2.3 SAF PARAMETER UPDATES

We made structural changes to the SAF parameters by converting **ceilingAmountPerCardType** and **haloAmountPerCardType** into **objects** and grouping **offline-related fields** into new request and response structures.

## 1. Converted **ceilingAmountPerCardType** and **haloAmountPerCardType** to Objects



Before:

```
// Create a Manage request object
ManageRequest manageRequest = new ManageRequest();
manageRequest.TransType = 26;

// Set values as Strings (no longer supported)
manageRequest.CeilingAmountPerCardType =
"ceiling_amount_per_card_type";
manageRequest.HaloPerCardType = "halo_per_card_type";
// Create a Manage request object
ManageRequest manageRequest = new ManageRequest();
manageRequest.TransType = 38;

// After the transaction response is received, retrieve data as
Strings
String ceilingAmountPerCardType =
manageResponse.ceilingAmountPerCardType;
String haloPerCardType = manageResponse.haloPerCardType;
```

After:

```
// Create a SetSAFParameters request object
SetSafParametersRequest setSafParametersRequest = new
SetSafParametersRequest();

// Create a CardAmount object and assign values
CardAmount cardAmount = new CardAmount();
cardAmount.setVisa = "500";

// Set the object to SAF parameters
setSafParametersRequest.setCeilingAmountPerCardType(cardAmount);
setSafParametersRequest.setHaloPerCardType(cardAmount);

// Create a GetSAFParameters request object
GetSafParametersRequest getSafParametersRequest = new
GetSafParametersRequest();

// After the transaction response is received, retrieve data as
CardAmount objects
CardAmount ceilingAmountPerCardType =
getSafParametersResponse.ceilingAmountPerCardType();
CardAmount haloPerCardType =
getSafParametersResponse.haloPerCardType();
```

## 2. Moved **startDateTime**, **endDateTime**, and **durationInDays** to **Offline Information** Groups

Before:

```
// Create a Manage request object
ManageRequest manageRequest = new ManageRequest();
manageRequest.TransType = 26;

// Set offline-related values directly in the request
manageRequest.StartDateTime = "20001231235959";
manageRequest.EndDateTime = "20001231235959";
manageRequest.DurationInDays = "10";
// Create a Manage request object
ManageRequest manageRequest = new ManageRequest();
manageRequest.TransType = 38;

// After the transaction response is received, retrieve values from response
String startDateTime = manageResponse.StartDateTime;
String endDateTime = manageResponse.EndDateTime;
String durationInDays = manageResponse.DurationInDays;
```

After:

```
// Create a SetSAFParameters request object
SetSafParametersRequest setSafParametersRequest = new
SetSafParametersRequest();

// Create an OfflineInformationRequest object and assign values
OfflineInformationRequest offlineInformation = new
OfflineInformationRequest();
offlineInformation.setStartOfflineDateTime("20001231235959");
offlineInformation.setEndOfflineDateTime("20001231235959");
offlineInformation.setDurationOfflineInDays("10");

// Assign to SAF parameters
setSafParametersRequest.setOfflineInformation(offlineInformation);

// Create a GetSAFParameters request object
GetSafParametersRequest getSafParametersRequest = new
GetSafParametersRequest();

// After the transaction response is received, retrieve data using
OfflineInformationResponse
OfflineInformationResponse offlineInformation =
getSafParametersResponse.offlineInformation();
String startOfflineDateTime =
offlineInformation.startOfflineDateTime();
String endOfflineDateTime = offlineInformation.endOfflineDateTime();
String durationOfflineInDays =
offlineInformation.durationOfflineInDays();
```

### PAX Customer Support

For questions or help with the terminal installation please contact your service provider or PAX customer support.

PAX Technology Inc.  
8775 Baypine Road  
Jacksonville, FL 32256

support@pax.us  
(877) 859-0099  
www.pax.us

This document is provided for informational purposes only. All features and specifications are subject to change without notice. The PAX name and PAX logo are registered trademarks of PAX Corporation and/or its affiliates. Other names may be trademarks of their respective owners. Copyright © 2024, PAX Technology Limited, all rights reserved.

***\*PAX Technology, Inc. is not responsible for the content, quality, accuracy, or completeness of any information or materials contained on these pages. PAX Technology, Inc. does not endorse any content, viewpoints, products, or services contained on these pages and shall not be held liable for any losses caused by reliance on the accuracy, reliability, or timeliness of such information. Any person or entity that relies on any information obtained from these pages does so at his/her own risk.***